

INTRODUCTION 1)

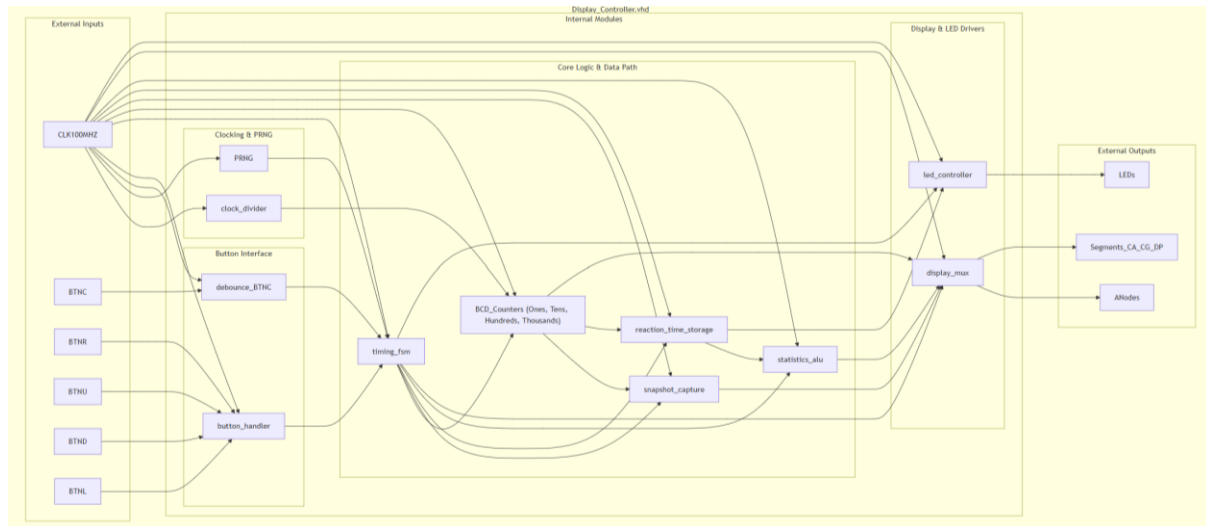
In this project, we implemented a reaction timer on a NEXYS 4-DDR FPGA device. The specifications for the design are summarised below:

- Prompt the user with a countdown in the form of three decimal places on the 7-segment display that turn off sequentially.
- Record the time between the final decimal place turning off and the user pushing the central BTNC button.
- Once the BTNC button has been pushed, the users reaction time in ms is displayed on the 7-segment display.
- LED0–1 show the number of stored times (0–3), and LED4/5/6 indicate average, maximum, or minimum display modes. An early BTNC press lights LED15, to indicate an early button press, a subsequent BTNC clears the error.
- When not recording a reaction time, the user can push:
 - ➔ The right BTNR button to display the average of the last three reaction times.
 - ➔ The up BTNU to display the longest reaction time.
 - ➔ The down BTND to display the users shortest reaction time.
 - ➔ The left BTNL button to clear the reaction time data.
- Additionally, a random number generator was implemented to vary the final decimal point turning off and beginning recording, making the reaction timer unpredictable and truly testing the user's reaction time.

The design achieves all of the specifications and succeeds in testing the users reaction time, providing them with data on their performance.

DESIGN SUMMARY AND JUSTIFICATION 2)

This section gives a summary of the methods and justifications behind the major techniques used to create the reaction timer. All files mentioned in this section can be found in the code appendix. Refer to the modularisation scheme in Graph 1 to understand the program structure.



Graph 1. Modularisation scheme for the reaction timer

The step counter makes use of each of the VHDL coding styles. Files such as the top-level display controller make large use of a structural VHDL code style. Using internal signal definitions and port map instantiations to connect the modules, creating a structure without defining module behaviour. Examples of structural code include Figure 1 in the appendix, which defines the slow clock signal, which, in line 157 of Figure 2, is passed to the counter for the milliseconds, forming the base for the timer. Figure 3 shows lines 296-303, showing the instantiation of the clock divider module, and how the signals/constants are linked to the inputs/outputs of the module. Structural code style was favoured here for three main reasons. Firstly, structural code creates an explicit hierarchy between files, allowing easy re-routing between or around behavioural blocks. Secondly, each module can be treated as a black box, allowing the development and testing of each module independently, speeding up development and implementation time. Finally, structural code works well for the high-level files, as the synthesis tools used by Vivado can correctly identify the intended code structure and implement it correctly.

The bcd_to_7_seg file uses mostly dataflow VHDL code style, using case statements to define the outputs from the module for a specific set of input cases. Lines 16-29 of Figure 4 demonstrate the fundamentals of dataflow style. For each of the finite inputs that can be passed to the module, a single output is assigned to each one of them, with all other states defaulting to the blank “1111111” state. Data flow style was used here because a multiplexer is inherently suited for a data flow approach, as there are a finite number of possible inputs, and one output for each input case. Additionally, data flow is the best way for this style of mass, direct assignment from input to output. Dataflow style improves readability, looking very similar to a truth table, as each input case is associated directly and exclusively with a single output case. Additionally, the process is sensitive to all the relevant signals and can immediately recalculate the output when input changes, a key feature in multiplexer behaviour.

The PRNG module is written almost entirely in behavioural style. An external clk signal governs when its *linear-feedback shift register (LFSR)* state is updated, so the code only needs to describe the algorithm rather than instantiate gates. At each rising edge of clk, lines 28 and 30 perform a logical shift-right (LSR) operation on the 16-bit lfsr_reg: the register is shifted (lfsr_reg(14 downto 0)) and the freshly-computed XOR feedback bit is appended at bit 0. This single LSR line captures the entire register movement and feedback insertion without any explicit wiring, which is why behavioural style is ideal here. Because the seed (x"ACE1") and tap positions are just variables/constants, changing the sequence length or starting state is a one-line edit—no schematic re-wiring required. Behavioural code also simulates exactly like software, so functional verification of the PRNG in a test-bench is fast and straightforward, even with its compact but non-trivial logic.

The implementation of the display FSM does not follow the requirements laid out in the project specifications. Instead of containing the ~300 lines of FSM logic into a single process statement, which would render the function unreadable, extendable and unable to be debugged, the FSM was decomposed into several smaller processes. For example, Figure 6 presents the RTL description of the reaction timer's input capture process, detailing the registers and combinational logic that generate the capture signal. Decomposing the FSM results in three main benefits for module performance. Firstly, each process handles only one area of logic (i.e next state logic, saved data clearing, and output decoding), improving code maintainability and reducing the chance of creating accidental interactions or inferred latches. Secondly, because the processes are all triggered on a rising clock edge, every input is serviced within a single clock cycle, eliminating cascaded if/else ladders, which delay response times. Finally, the increased modularisation of the FSM simplifies the simulation and the creation of targeted testbench simulations, allowing independent resets or initialisations to each FSM process if needed. Overall, splitting the FSM into multiple sub-processes makes code clearer, timing deterministic and verification easier.

MODULE TESTING 3)

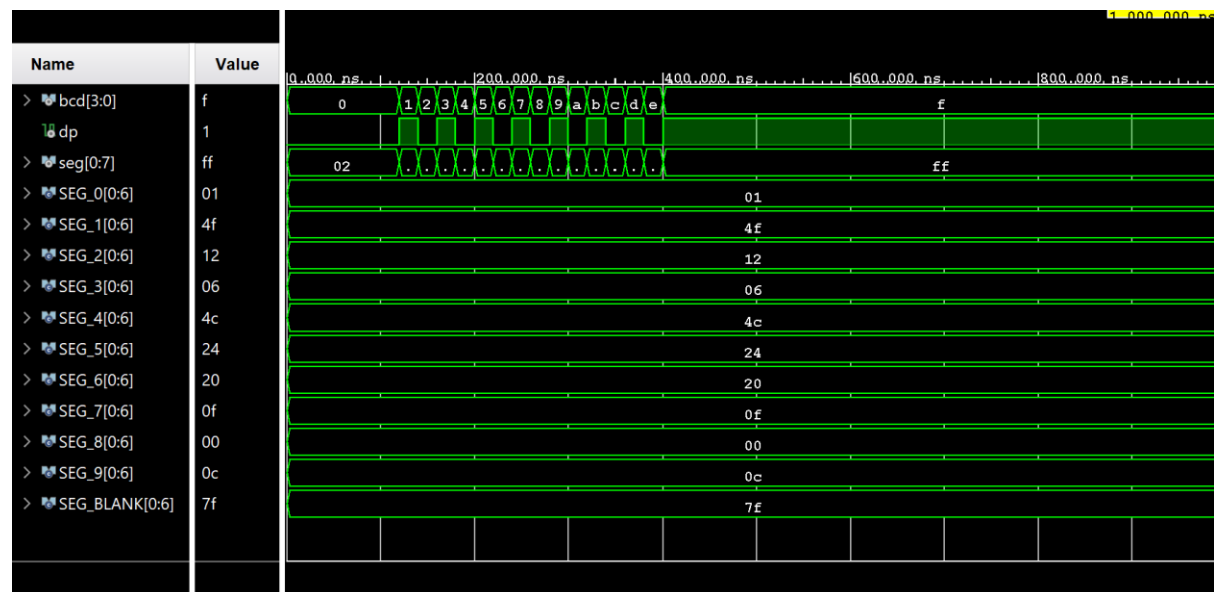
This section focuses on the testing of the BCD_TO_7_SEG module, all figure references can be found in the code appendix.

While all design modules were tested thoroughly with test benches and hardware testing, this section will focus on the BCD_TO_7_SEG module, which decodes the binary coded decimal output from the clock into a binary sequence that can be used to drive the cathodes of the 7-segment display, including the decimal point.

Firstly, test signals, which take the place of the connections to other modules and hardware, are created so module inputs and outputs can be monitored. Figure 7 shows the assignment of the signals, ‘bcd’ standing in for the Binary Coded Decimal input from module 5, ‘dp’ and ‘seg’ representing the signals passed to drive the LEDs. These signals are assigned to the inputs/outputs of the module, as seen in Figure 8. This assignment allows monitoring during module testing, and for simplicity, the test signals were given the same names as the ports they were associated with.

In the code appendix, Figure 9 shows the loop used to test the module, including the logic dictating the states, assignments and real-time considerations of the test function. Inside the loop dictating the behaviour of the module test, an incrementing 4-bit logic vector is assigned to ‘bcd’, which is decoded by the module and outputted to the ‘seg’ signal. After the loop is halfway completed, the decimal place is set active and is reflected in the ‘seg’ output from the module. Due to the real-time nature of the test, the loop is delayed by 100ns to allow proper observation of the waveforms

Graph 2 shows the simulation waveforms that confirm the decoder behaves exactly as intended.

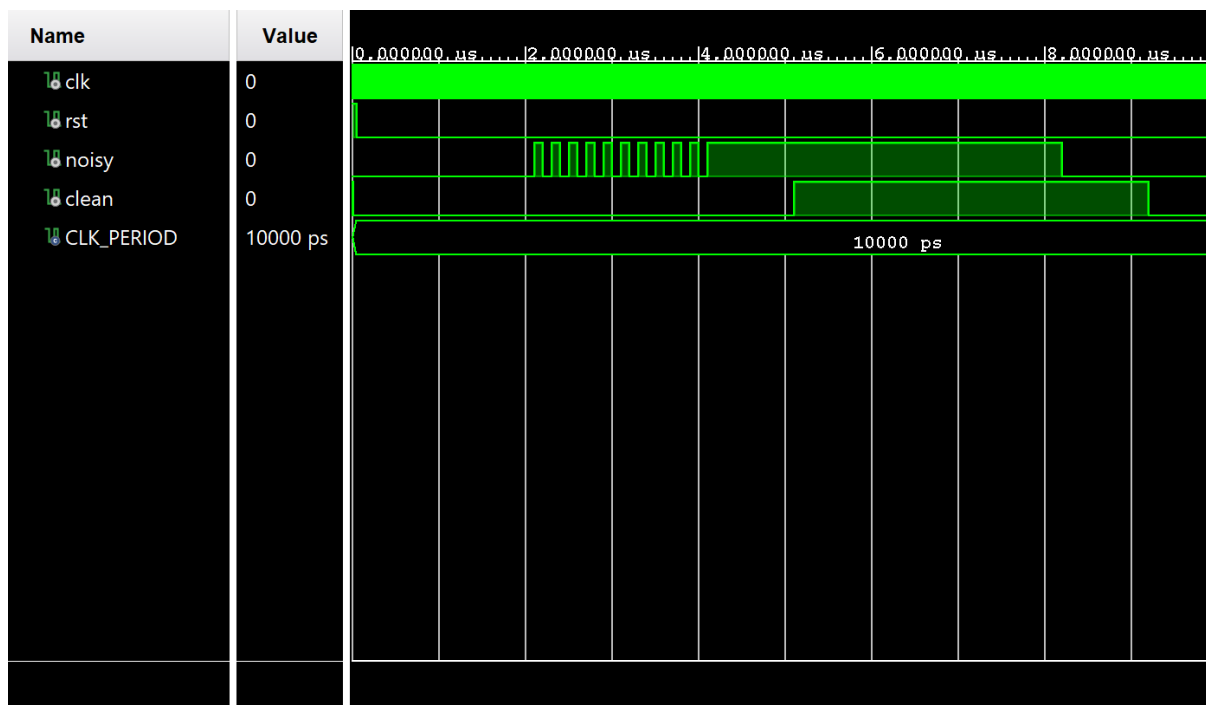


Graph 2. bcd_to_7_seg_tb, simulation output waveform

From Graph 2, as the BCD input is stepped from 0 to 9, each segment line (CA–CG) adopts the correct pattern—hex 01, 4F, 12, ..., 0C—while inputs A–F all yield the blank pattern (7F). The MSB (seg(7)) mirrors the dp signal precisely, producing FF when the display is blank with DP active, and 7F when blank with DP inactive. Notably, no spurious glitches or intermediate values appear on any segment lines during transitions, demonstrating a complete sensitivity list and the absence of unintended latches. These results in Graph 2p give confidence that the decoder will drive a real seven-segment display cleanly and without timing hazards.

DESIGN/IMPLEMENTATION PROBLEMS 4)

A design implementation problem we encountered was ensuring the debounce circuit would reliably suppress the rapid, unpredictable chatter of a mechanical push-button without delaying genuine presses. To diagnose and fine-tune its behaviour, we developed a VHDL testbench (see Appendix Figure 10 whose stimulus process toggles the noisy input every 100 ns for twenty cycles—emulating switch bounce—and immediately asserts that clean remains ‘0’ throughout. To avoid prohibitively long simulation times, we parameterized the debounce counter as a generic and in the testbench, overrode it to just 100 cycles ($\sim 1\text{ }\mu\text{s}$) while retaining the 1,000,000-cycle (10 ms) default for hardware. The testbench then holds noisy high beyond this threshold and checks that clean issues exactly one rising edge after the interval, and likewise one falling edge on release. Graph 3 shows the simulation waveforms confirming that chatter is ignored and only a genuine press is detected.



Graph 3. debounce_tb simulation output waveform

This simulation-driven validation confirmed that all sub-threshold chatter is ignored and that genuine presses still produce prompt, single-cycle clean transitions, giving us confidence the denouncer will behave correctly in hardware.

CONCLUSION 5)

Through this project we deepened our understanding of modular VHDL design by breaking the reaction timer into discrete, purpose-built components—clock divider, debounce circuit, PRNG, BCD-to-7-segment decoder, FSM, storage, statistics ALU, display multiplexer, and LED controller—each with a clear interface and single responsibility. This hierarchical approach not only made individual modules easier to develop and simulate in isolation (for example, exercising the debounce logic with a tailored testbench) but also simplified top-level integration: swapping or reusing modules required minimal changes to the structural architecture. Parameterizing the debounce interval as a generic proved especially powerful: it let us speed up simulation by overriding the default 1,000,000-cycle threshold to just 100 cycles, while retaining the real 10 ms behavior in hardware. Throughout, assertion-driven, waveform-based verification caught subtle issues—missing sensitivity entries or off-by-one counter resets—long before synthesis, underscoring the value of a simulation-first workflow.

Equally important was our exploration of data-flow versus structural coding styles. In the data-flow approach, the BCD-to-7-segment decoder is implemented as a concise case statement that directly maps each input value to its output pattern, greatly improving readability and maintainability. By contrast, the FSM is built structurally—wiring together clearly named state-transition signals—which ensures deterministic sequencing and makes timing behaviour straightforward to verify. Careful management of each process’s sensitivity list prevented unintended latches and produced glitch-free operation throughout. Together, these techniques—modular components, generics, clear signal naming, and disciplined sensitivity lists—delivered a robust, easy-to-debug reaction timer and a proven blueprint for future FPGA projects.

Looking ahead, several enhancements could further elevate the reaction-timer system. Integrating a UART interface to stream each reaction time in real-time to a host PC would enable comprehensive logging and advanced post-run analysis. Refining the error-state feedback—such as displaying a more clear error message on the seven segment displays would help users understand and correct invalid presses. Finally, adding an audible “go” cue via a simple buzzer would enrich the user experience by combining visual and audio prompts. Together, these improvements would deepen both the system’s usability and its value as an educational FPGA design example.

APPENDICES & REFERENCES 6)

DESIGN DESCRIPTION FIGURES

```
37 | -----
38 | -- Slow Clock Signal for 1-ms ticks
39 | -----
40 | signal slow_clk_ms : std_logic;
```

Figure 1. Initialisation of the internal slow clock signal

```
150 | -----
151 | -- 1) Clock Divider for 1 ms
152 | -----
153 | clock_divider_ms_inst : entity work.clock_divider
154 |     port map (
155 |         CLK      => CLK100MHZ,
156 |         UPPERBOUND => MS_UPPERBOUND,
157 |         SLOWCLK   => slow_clk_ms
158 |     );
```

Figure 2. Instantiation of the clock divider module

```
293 | -----
294 | -- 7) Ones Digit Counter (1ms units, clocked by 1kHz slow clock)
295 | -----
296 | ones_counter_inst : entity work.Ones_counter -- Entity name from Millisecond_Counter.vhd
297 |     port map (
298 |         CLK_1MS  => slow_clk_ms, -- Use the 1ms clock
299 |         RESET    => counter_rst,
300 |         EN       => counter_en, -- Use FSM enable
301 |         COUNT    => ones_count,
302 |         ROLLOVER => ones_rollover
303 |     );
```

Figure 3. Linking of the milliseconds counter input/outputs to internal signals

```
16 | case bcd is
17 |     when "0000" => seg(0 to 6) <= "0000001"; -- 0
18 |     when "0001" => seg(0 to 6) <= "1001111"; -- 1
19 |     when "0010" => seg(0 to 6) <= "0010010"; -- 2
20 |     when "0011" => seg(0 to 6) <= "0000110"; -- 3
21 |     when "0100" => seg(0 to 6) <= "1001100"; -- 4
22 |     when "0101" => seg(0 to 6) <= "0100100"; -- 5
23 |     when "0110" => seg(0 to 6) <= "0100000"; -- 6
24 |     when "0111" => seg(0 to 6) <= "0001111"; -- 7
25 |     when "1000" => seg(0 to 6) <= "0000000"; -- 8
26 |     when "1001" => seg(0 to 6) <= "0001100"; -- 9
27 |     when "1010" => seg(0 to 6) <= "1111111"; -- Blank
28 |     when others => seg(0 to 6) <= "1111111"; -- Default: Blank
29 | end case;
```

Figure 4. Case statement used by the BCD_TO_7_SEG multiplexer

```

16 architecture Behavioral of PRNG is
17     signal lfsr_reg : std_logic_vector(15 downto 0) := x"ACE1"; -- Non-zero initial seed
18 begin
19
20     process(clk, reset)
21         variable feedback : std_logic;
22     begin
23         if reset = '1' then
24             lfsr_reg <= x"ACE1"; -- Reset to initial seed
25         elsif rising_edge(clk) then
26             if enable = '1' then
27                 -- Calculate feedback bit (XOR of taps)
28                 feedback := lfsr_reg(15) xor lfsr_reg(13) xor lfsr_reg(12) xor lfsr_reg(10);
29                 -- Shift register and insert feedback
30                 lfsr_reg <= lfsr_reg(14 downto 0) & feedback;
31             end if;
32         end if;
33     end process;

```

Figure 5. Behavioural style code for the Pseudo Random Number Generator

```

378 -----
379 -- 5) Snapshot Trigger: pulses when going from timing -> display_clear
380 -----
381 process(clk)
382 begin
383     if rising_edge(clk) then
384         if (current_state = timing and next_state = display_clear) then
385             snapshot <= '1';
386         else
387             snapshot <= '0';
388         end if;
389     end if;
390 end process;

```

Figure 6. Finite State Machine process for input capture

TESTING FIGURES

```

11 signal bcd : std_logic_vector(3 downto 0) := (others => '0');
12 signal dp  : std_logic := '0';
13 signal seg : std_logic_vector(0 to 7);

```

Figure 7: Test signals for BCD_TO_7_SEG

```

16 -- Instantiate the Unit Under Test (UUT)
17 UUT: entity work.bcd_to_7seg
18     port map(
19         bcd => bcd,
20         dp  => dp,
21         seg => seg
22     );

```

Figure 8. Signal mapping for BCD_TO_7_SEG testing

```

24      -- Stimulus process: cycle through all 4-bit BCD values (0..15)
25      stimulus_proc: process
26      begin
27          for i in 0 to 15 loop
28              -- Apply the 4-bit pattern to bcd
29              bcd <= std_logic_vector(to_unsigned(i, 4));
30
31              -- Toggle the decimal point at halfway
32              if i < 8 then
33                  dp <= '0';
34              else
35                  dp <= '1';
36              end if;
37
38              wait for 20 ns; -- Wait time between steps
39          end loop;
40
41          wait; -- Stop simulation
42      end process stimulus_proc;

```

Figure 9. Test loop for BCD_TO_7_SEG

```

66      for i in 0 to 20 loop
67          :      noisy <= not noisy;
68          :      wait for 100 ns;
69      end loop;
70      -- still suppressed
71      wait for 2 us;
72      assert clean = '0'
73          report "Clean went high during bounce" severity error;
74
75      -- 4) Valid press: hold high longer than debounce window
76      noisy <= '1';
77      wait for 2 us;
78      assert clean = '1'
79          report "Clean did not go high after valid press" severity error;
80
81      -- 5) Release: back to low beyond debounce window
82      noisy <= '0';
83      wait for 2 us;
84      assert clean = '0'
85          report "Clean did not return low after release" severity error;
86
87      report "Debounce testbench passed" severity note;
88      wait;

```

Figure 10. Testbench excerpt toggling noisy every 100 ns to emulate bounce and asserting that clean only transitions once on a valid press and release.